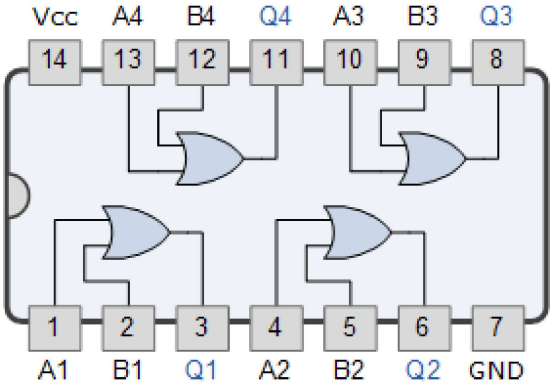
Також існує **7432** — чотири 2-входові логічні елементи OR у TTL-виконанні.



Логічний елемент NOT

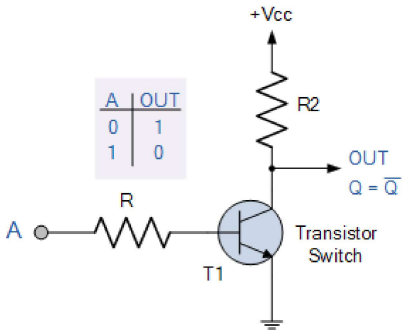
Логічний елемент NOT — це **найпростіший з усіх логічних елементів**, який часто називають **інверсним буфером** або просто **інвертором**.

Інверсні логічні елементи NOT — це **одновходові пристрої**, вихід яких зазвичай перебуває на **логічному рівні "1"**, і переходить у **логічний "0" (LOW)**, коли вхід має **логічний "1"**. Іншими словами, він **інвертує** (доповнює) вхідний сигнал.

Вихід елемента NOT знову стає **високим (HIGH)** тільки тоді, коли його вхід перебуває на **логічному рівні "0"**. **Булевий вираз**: $\overline{A} = Q$ **"Якщо A не істинне, тоді Q істинне"**

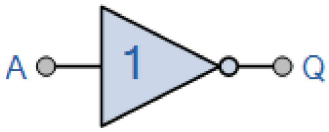
Логічний елемент NOT на транзисторі

Простий одновходовий логічний елемент NOT можна побудувати за допомогою **резисторно-транзисторної логіки (RTL)**, де вхід підключено **безпосередньо до бази транзистора**. Для отримання інверсного сигналу на виході Q транзистор має бути або **повністю увімкнений (ON)**, або **повністю вимкнений (OFF)**.



Логічні елементи NOT доступні у вигляді **цифрових схем**, які реалізують потрібну логічну функцію. Стандартне графічне позначення **елемента NOT** — це **трикутник, спрямований вправо, із кружечком на виході**. Цей кружечок називають **"бульбашкою інверсії"** (*inversion bubble*).
Він використовується в символах **NOT**, **NAND** і **NOR** на виході для позначення виконання **логічної операції інверсії**.

Бульбашка інверсії вказує на **інверсію (доповнення) сигналу** й може розташовуватись як **на виході**, так і **на вході**, або на обох одночасно.

Позначення	**Таблиця істинності**	
 Інвертор (логічний елемент NOT)	**A**	**Q**
	0	1
	1	0
Булевий вираз Q = не A або $\overline{A}$	Читається як: інверсія A дає Q	

Логічні елементи NOT формують **доповнення (інверсію) вхідного сигналу**. Свою назву вони отримали тому, що коли вхід перебуває в стані **“1” (HIGH)**, вихід **не буде “1”**, а коли вхід — **“0” (LOW)**, вихід **не буде “0”**.

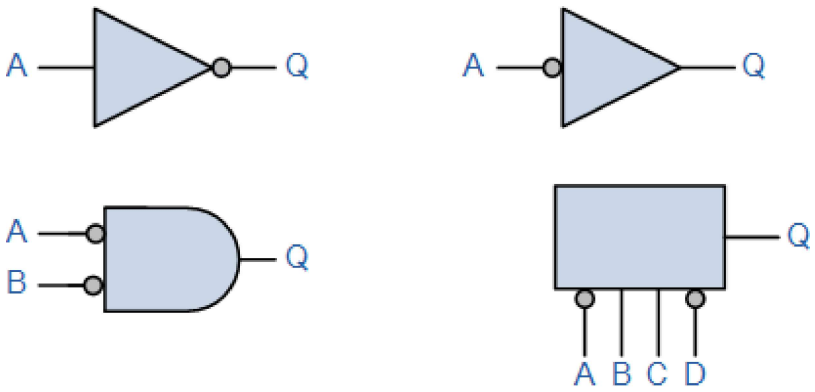
Оскільки це **одновходові пристрої**, елементи NOT зазвичай **не відносять до пристроїв “прийняття рішень”** або повноцінних “воріт” на кшталт AND чи OR, які мають два й більше входів. **Промислові мікросхеми NOT** випускаються у вигляді корпусів, що містять **4 або 6 інверторів**.

“Бульбашка” (о), розташована на виході символу NOT, означає **інверсію (доповнення) вихідного сигналу**. Вона також може бути розташована **на вході елемента**, вказуючи на **активний низький рівень входу (active-LOW)**.

Інверсія вхідного сигналу **не обмежується елементом NOT** і може застосовуватись у будь-якій цифровій схемі або логічному елементі — принцип роботи інверсії **однаковий**, чи вона стоїть на вході, чи на виході. Найпростіше запам’ятати: **бульбашка = інвертор**.

Інверсія сигналу з використанням активного низького рівня (Active-low Input Bubble)

Коли бульбашка позначена **на вході**, елемент або схема сприймають **логічний “0” як активний сигнал**, і виконують свою дію саме при низькому рівні.



Позначення “бульбашкою” для інверсії входу

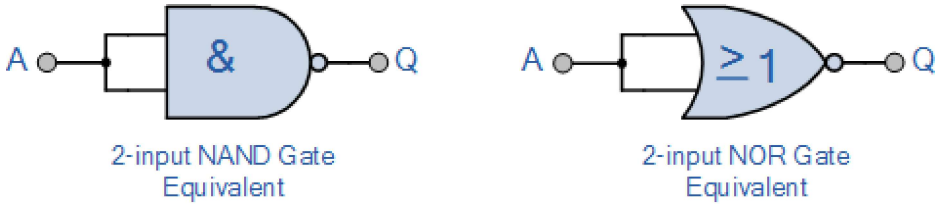
Еквіваленти на основі NAND та NOR

Інвертор (**логічний елемент NOT**) можна також реалізувати, використовуючи **стандартні логічні елементи NAND або NOR**. Для цього **всі входи елемента з’єднують разом** і підключають до **спільного вхідного сигналу**.

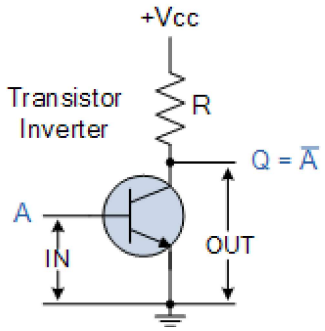
Таким чином:

- **NAND** із усіма об’єднаними входами працює як **NOT**
- **NOR** із усіма об’єднаними входами теж працює як **NOT**

Цей прийом часто застосовують, коли в схемі **немає окремого інвертора**, але є **зайвий NAND або NOR**.



Дуже простий інвертор можна також зробити, використовуючи лише **одноступеневий транзисторний ключовий каскад**, як показано на схемі.



Коли на вхід бази транзистора в точці **“A”** подається високий рівень (**HIGH**), транзистор **відкривається**, і через нього протікає **струм колектора**, створюючи **падіння напруги на резисторі R**. У результаті **вихідна точка “Q”** з’єднується із землею, що призводить до **нульової напруги на виході Q**.

Коли на вхід бази транзистора в точці **“A”** подається низький рівень (**0 В**), транзистор **вимикається (OFF)**, і через резистор **струм колектора не протікає**. У результаті на виході **Q** з’являється **високий рівень**, близький до **+Vcc**.

Отже:

- При **високій напрузі на вході A → вихід Q низький (LOW)**
- При **низькій напрузі на вході A → вихід Q високий (HIGH)**

Таким чином формується **доповнення (інверсія) вхідного сигналу**.

Шестивходові інвертори Шмітта (Hex Schmitt Inverters)

Звичайний **інвертор** або логічний елемент **NOT** зазвичай реалізується на **транзисторних ключових схемах**, які **не перемикаються миттєво** з одного стану в інший — завжди існує певна **затримка перемикання**. Також, оскільки транзистор є **підсилювачем струму**, він може працювати і в **лінійному режимі**, тож **невеликі зміни рівня вхідної напруги** спричинять зміни на виході або навіть **множинні випадкові перемикання** за наявності шуму в колі.

Щоб **усунути ці проблеми**, використовують **інвертор Шмітта (Schmitt Inverter)** або **Hex Inverter**.

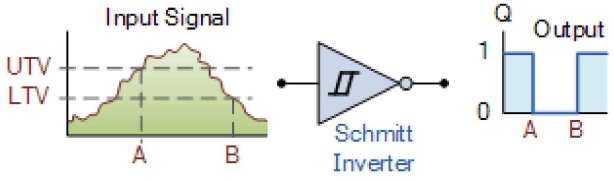
Особливості роботи інвертора Шмітта

Ми знаємо, що всі цифрові логічні елементи працюють із **двома логічними рівнями**:

- **Логічна “1”**: для TTL вхідна напруга **2,0–5,0 В**
- **Логічний “0”**: для TTL вхідна напруга **нижче 0,8 В**

Інвертор Шмітта спрацьовує (перемикає стан), коли вхідний сигнал **перевищує верхній поріг напруги (UTV)** — тоді вихід змінюється на **LOW** і залишається в цьому стані, поки вхідний сигнал **не опуститься нижче нижнього порогу напруги (LTV)** — тоді вихід стає **HIGH**.

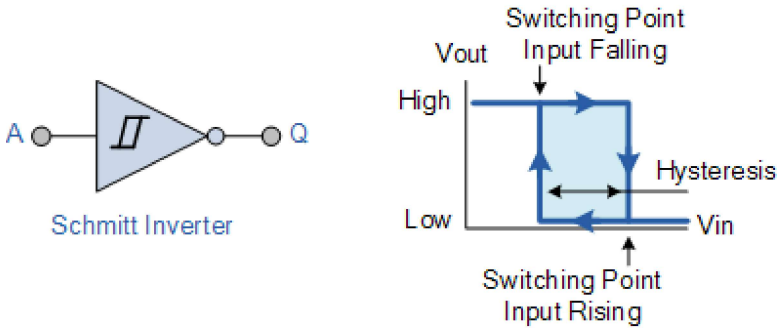
Іншими словами, **інвертор Шмітта має гістерезис** у своїй схемі перемикання, що робить його стійким до шумів і повільних змін сигналу.



Ця дія перемикання між **верхнім і нижнім пороговими рівнями** забезпечує **набагато чистіший і швидший сигнал перемикання “ON/OFF”**, що робить інвертор Шмітта **ідеальним для обробки сигналів з повільним зростанням або спаданням**.

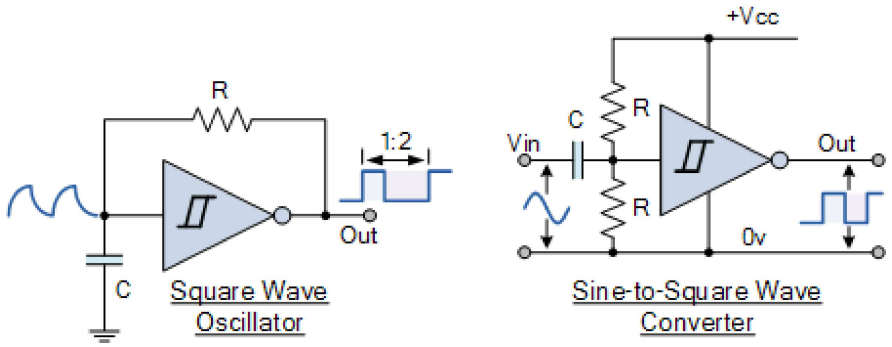
Завдяки цьому **тригер Шмітта** можна використовувати для **перетворення аналогових сигналів у цифрові**, як показано на схемі.

Інвертор Шмітта



Дуже корисним застосуванням **інверторів Шмітта** є їх використання як **генераторів** або **перетворювачів синусоїдального сигналу у прямокутний**, що дозволяє отримувати **тактуючі імпульси (square wave clock signals)**.

Генератор на інверторі NOT з тригером Шмітта



Перший приклад схеми показує **дуже простий малопотужний RC-генератор** на базі інвертора Шмітта, який генерує **прямокутний сигнал**. На початку конденсатор **C** повністю розряджений, тому вхід інвертора перебуває в стані **“LOW”**, а його вихід — **“HIGH”**. Оскільки вихід інвертора з’єднаний із його входом і конденсатором через резистор **R**, конденсатор починає заряджатися.

Коли напруга на конденсаторі досягає **верхнього порогу спрацювання інвертора**, інвертор змінює стан, вихід стає **“LOW”**, і конденсатор починає розряджатися через резистор, поки не досягне **нижнього порогу**, після чого інвертор знову перемикається. Таке чергування станів формує **прямокутний сигнал** із коефіцієнтом заповнення близько **33%**, частота якого обчислюється за формулою:

$$f = \frac{680}{R * C}$$

Друга схема **перетворює синусоїдальний сигнал** (або будь-який інший змінний сигнал) у **прямокутний**. Вхід інвертора підключено до точки дільника напруги, який задає робочу точку схеми. Вхідний конденсатор блокує постійну складову, пропускаючи лише змінну.

Коли вхідний сигнал перевищує **верхній** або опускається нижче **нижнього порогу** інвертора, вихід перемикається між **“HIGH”** та **“LOW”**, формуючи прямокутні імпульси. Ця схема створює вихідний імпульс на **позитивному фронті** вхідної хвилі, але якщо підключити **другий інвертор Шмітта** на виході першого, можна отримати імпульс на **негативному фронті** сигналу.

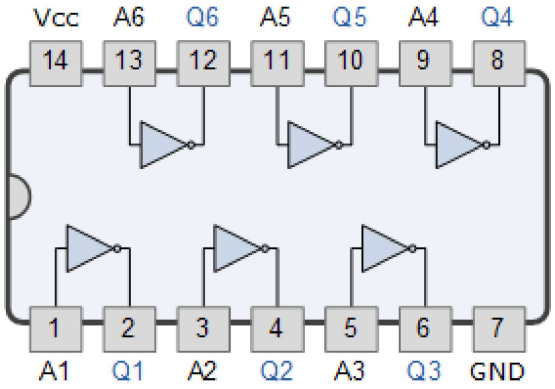
Поширені мікросхеми логічних елементів NOT та інверторів

TTL-логіка:

- **74LS04** — шість інверторів
- **74LS14** — шість інверторів Шмітта
- **74LS1004** — шість інверторних драйверів

CMOS-логіка:

- **CD4009** — шість інверторів
- **CD4069** — шість інверторів
- **7404** — стандартний інвертор (NOT)



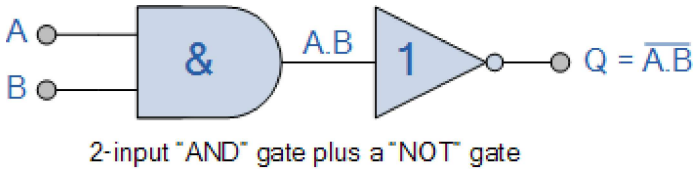
Логічний елемент NAND

Логічний елемент NAND — це комбінація **цифрового логічного елемента AND** та **елемента NOT**, з'єднаних послідовно. Тобто, спочатку виконується **логічне множення (AND)** всіх вхідних сигналів, а потім результат **інвертується (NOT)**. У булевій формі: $Q = \overline{A * B}$

Це означає, що **NAND дає логічний "0" лише тоді, коли всі його входи = "1"**, а в усіх інших випадках вихід буде **"1"**.

Елемент **NAND** (*Not – AND*) має вихід, який **звичайно перебуває на логічному рівні "1"** і переходить у **"0" (LOW)** лише тоді, коли **всі його входи перебувають на логічному рівні "1"**. Логічний елемент **NAND** є **зворотною (доповняльною) формою** елемента **AND**, який ми розглядали раніше.

Еквівалентність логічного елемента NAND:



2-input "AND" gate plus a "NOT" gate

Булевий вираз для **логічного елемента NAND** базується на **логічному множенні**, але на відміну від елемента AND, він виконує його над **доповненнями входів**.

Його **булеве позначення**:

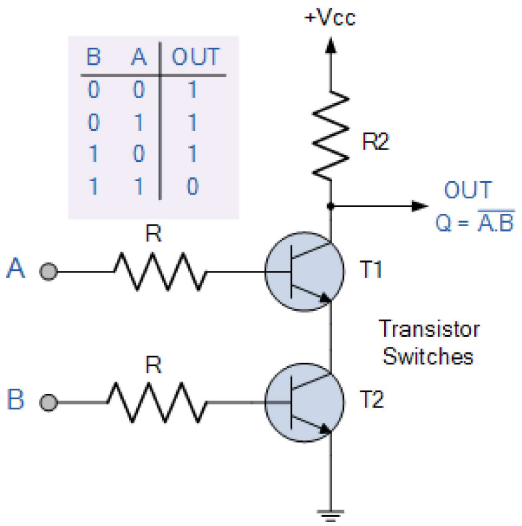
- використовується **крапка (·)** для позначення множення (AND)
- над усім виразом ставиться **риска (¯)**, яка означає **NOT (логічне заперечення)**

Отже, для 2-входового елемента NAND: $Q = \overline{A.B}$ Це означає, що вихід **Q** дорівнює **інверсії добутку входів A і B**.

Отже, роботу **2-входового цифрового елемента NAND** можна визначити так: **"Якщо і A, і B істинні, тоді Q — не істинне."**

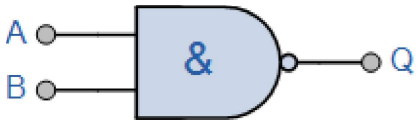
Транзисторний елемент NAND

Простий **2-входовий елемент NAND** можна побудувати за допомогою **резисторно-транзисторної логіки (RTL)**, з'єднавши транзисторні ключі, як показано на схемі. Входи підключаються **безпосередньо до баз транзисторів**. Для того щоб на виході **Q** був логічний "1", **хоча б один із транзисторів повинен бути закритий (OFF)**.



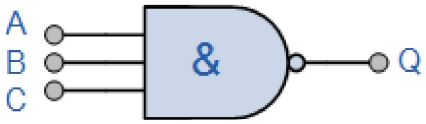
Логічні елементи NAND реалізуються у вигляді **цифрових схем**, які виконують потрібну логічну функцію, та мають **умовне графічне позначення** у формі **стандартного символу AND** з **кружечком ("бульбашкою інверсії")** на виході. Ця **бульбашка** позначає **інверсію** і відповідає символу елемента NOT.

Цифровий логічний елемент "NAND"

Позначення	Таблиця істинності		
 2-входовий логічний елемент NAND	**B**	**A**	**Q**
	0	0	1
	0	1	1
	1	0	1
	1	1	0

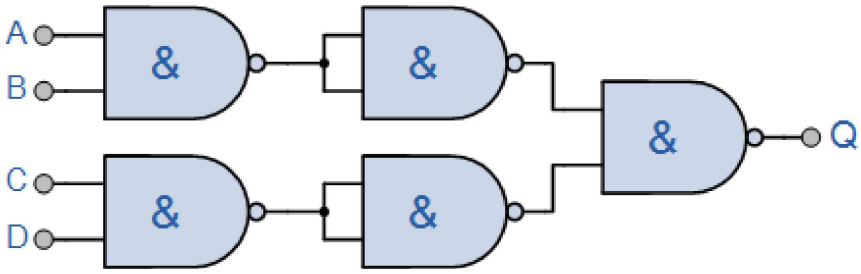
Булевий вираз $Q = \overline{A.B}$	Читається як: A і B дає **НЕ** Q

3-входовий логічний елемент AND

Позначення	**Таблиця істинності**			
	**C**	**B**	**A**	**Q**
	0	0	0	1
3-входовий логічний елемент NAND	0	0	1	1
	0	1	0	1
	0	1	1	1
	1	0	0	1
	1	0	1	1
	1	1	0	1
	1	1	1	0
Булевий вираз $Q = \overline{A.B.C}$	Читається як: A і B і C дає **НЕ** Q			

Як і у випадку з функцією **AND**, функція **NAND** може мати **будь-яку кількість входів**. У промислових умовах мікросхеми NAND зазвичай випускаються у виконанні **на 2, 3 або 4 входи**. Якщо потрібна більша кількість входів, можна **каскадувати стандартні елементи NAND**, щоб отримати потрібну кількість входів.

4-входова функція



Булевий вираз для цього **4-входового логічного елемента NAND** такий:

$$Q = \overline{A * B * C * D}$$

Якщо потрібна **непарна кількість входів**, будь-які **невикористані** можна утримувати у стані **логічної "1"**, підключивши їх безпосередньо до джерела живлення через відповідні **підтягувальні резистори (pull-up)**.

Функція штриха Шеффера

Функцію логічного елемента NAND іноді називають **функцією штриха Шеффера**. Вона позначається **вертикальною рискою (штрих)** або **стрілкою вгору**, наприклад:

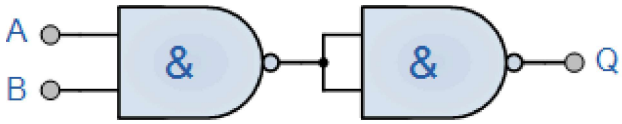
$A\ NAND\ B = A \mid B$ або $A \uparrow B$

Функція штриха Шеффера (NAND) використовується практично всюди в цифровій електроніці, оскільки є **універсальною логічною операцією**. Вона дозволяє будувати **будь-яку іншу логічну функцію** (AND, OR, NOT, XOR, NOR) лише з одного типу елементів.

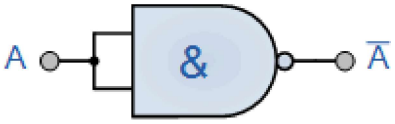
"Універсальний" елемент NAND

Елемент NAND вважається **універсальним**, оскільки є одним із найпоширеніших типів логічних елементів. Він може бути використаний для реалізації **будь-якої іншої логічної функції**, і на практиці саме NAND часто стає основою більшості цифрових схем. Об'єднуючи NAND у різних комбінаціях, можна отримати три базові функції: **AND**, **OR** та **NOT**, використовуючи **тільки NAND-елементи**.

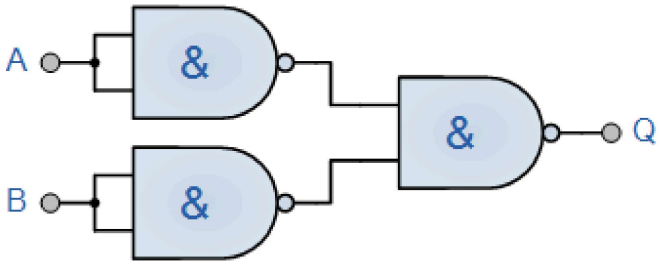
Різновиди логічних елементів



2-input AND Gate



NOT Gate



2-input OR Gate

Окрім трьох основних типів, з **окремих елементів NAND** можна побудувати **XOR (Виключне АБО)**, **XNOR (Виключне НЕ-АБО)** та навіть стандартний **NOR**.

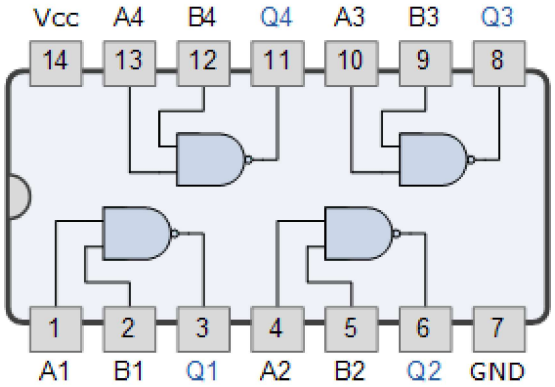
Поширені мікросхеми логічних елементів NAND

TTL-логіка:

- **74LS00** — чотири 2-входові NAND
- **74LS10** — три 3-входові NAND
- **74LS20** — два 4-входові NAND
- **74LS30** — один 8-входовий NAND

CMOS-логіка:

- **CD4011** — чотири 2-входові NAND
- **CD4023** — три 3-входові NAND
- **CD4012** — два 4-входові NAND
- **7400** — чотири 2-входові NAND (TTL-сумісна)

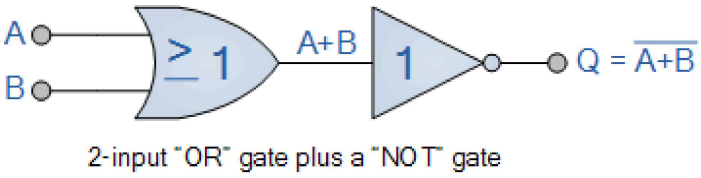


Логічний елемент NOR

Логічний елемент **NOR** — це комбінація цифрового логічного елемента **OR** та інвертора **NOT**, з'єднаних послідовно.

Інклюзивний елемент **NOR** (*Not-OR*) має вихід, який зазвичай перебуває на **логічному рівні "1"** і переходить у **"0" (LOW)**, якщо **будь-який із його входів** перебуває на логічному рівні "1". Логічний елемент **NOR** є **зворотною (доповняльною) формою** інклюзивного елемента **OR**, який ми розглядали раніше.

Еквівалент логічного елемента NOR



2-input "OR" gate plus a "NOT" gate

Булевий вираз для **логічного елемента NOR** відповідає **логічному додаванню (OR)**, яке виконується над **доповненнями входів**. Вираз позначається **знаком "+"** з **рискою зверху (¯)**, що означає **NOT** або логічне заперечення:

$$\overline{A + B} = Q$$

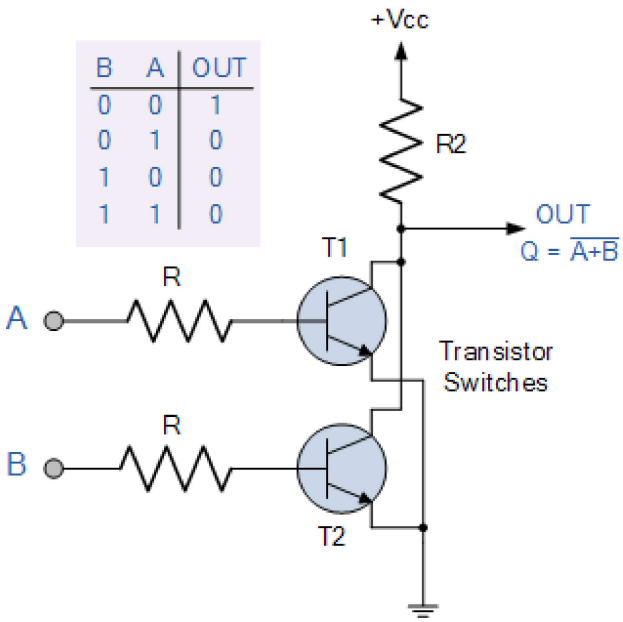
Отже, роботу **2-входового логічного елемента NOR** можна визначити так:

"Якщо A або B істинні, тоді Q — не істинне."

"Якщо і A, і B не істинні, тоді Q істинне."

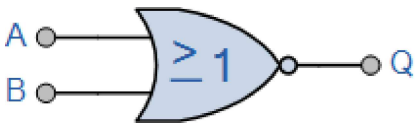
Транзисторний логічний елемент NOR

Простий **2-входовий логічний елемент NOR** можна побудувати на основі **резисторно-транзисторної логіки (RTL)**, де входи підключаються **безпосередньо до баз транзисторів**. Для отримання **логічної "1" на виході Q** обидва транзистори мають бути **закриті (OFF)**.



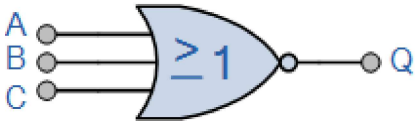
Логічні елементи NOR реалізуються у вигляді **цифрових схем**, що виконують потрібну логічну функцію. Їхнє **умовне графічне позначення** має форму **стандартного символу OR** з **кружечком ("бульбашкою інверсії")** на виході, який позначає **інверсію (NOT)**.

Цифровий логічний елемент "NOR"

Позначення	Таблиця істинності		
	B	A	Q
	0	0	1

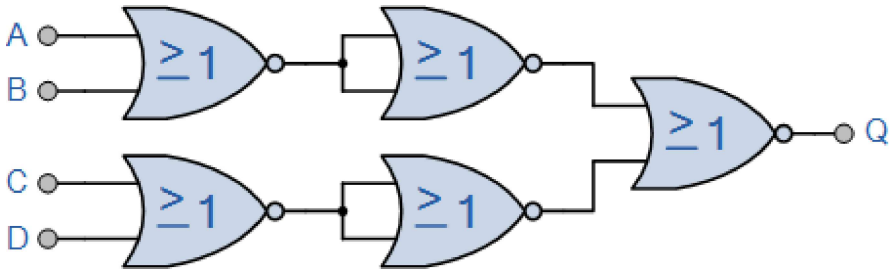
2-входовий логічний елемент NOR	0	1	0
	1	0	0
	1	1	0
Булевий вираз $Q = \overline{A + B}$	Читається як: A або B дає **НЕ** Q		

3-входовий логічний елемент NOR

Позначення	**Таблиця істинності**			
	**C**	**B**	**A**	**Q**
	0	0	0	1
3-входовий логічний елемент NOR	0	0	1	0
	0	1	0	0
	0	1	1	0
	1	0	0	0
	1	0	1	0
	1	1	0	0
	1	1	1	0
Булевий вираз $Q = \overline{A + B + C}$	Читається як: A або B або C дає **НЕ** Q			

Як і у випадку з функцією **OR**, функція **NOR** може мати **будь-яку кількість входів**. У промислових мікросхемах NOR зазвичай випускаються у виконанні **на 2, 3 або 4 входи**. Якщо потрібна більша кількість входів, можна **каскадувати стандартні елементи NOR**, щоб отримати необхідну кількість входів.

4-входова функція NOR



Булевий вираз для цього **4-входового елемента NOR** такий:

$$Q = \overline{A + b + C + D}$$

Якщо потрібна **непарна кількість входів**, будь-які **невикористані входи** можна утримувати в стані **логічного “0”**, підключивши їх безпосередньо до землі через відповідні **підтягувальні резистори (pull-down)**.

Функція Пірса

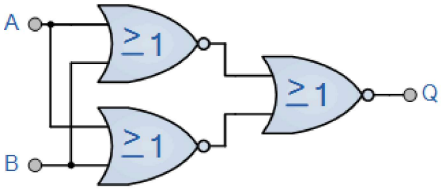
Функцію логічного елемента NOR іноді називають **функцією Пірса** (*Pierce Function*). Вона позначається **стрілкою вниз (↓)**, наприклад:

$$A \downarrow B$$

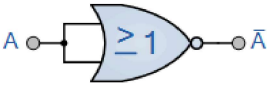
“Універсальний” елемент NOR

Подібно до NAND, елемент NOR є **універсальним** — з нього можна побудувати **будь-яку іншу логічну функцію**. Об’єднуючи NOR у різних комбінаціях, можна реалізувати **AND**, **OR** та **NOT**, використовуючи лише NOR-елементи.

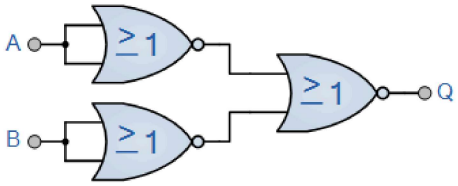
Різні логічні елементи, побудовані лише на основі NOR



2-input OR Gate



NOT Gate



2-input AND Gate

Окрім трьох основних типів, за допомогою **окремих елементів NOR** можна побудувати також **XOR (Виключне АБО)**, **XNOR (Виключне НЕ-АБО)** та стандартний **NOR**.

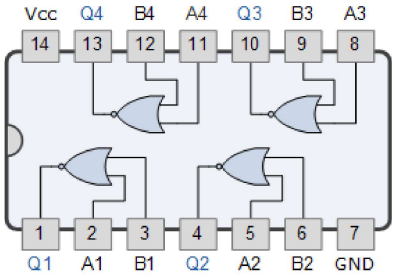
Поширені мікросхеми логічних елементів NOR

TTL-логіка:

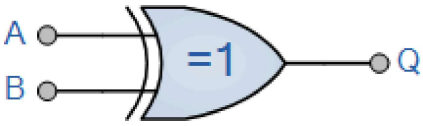
- **74LS02** — чотири 2-входові NOR
- **74LS27** — три 3-входові NOR
- **74LS260** — два 5-входові NOR

CMOS-логіка:

- **CD4001** — чотири 2-входові NOR
- **CD4025** — три 3-входові NOR
- **CD4002** — два 4-входові NOR
- **7402** — чотири 2-входові NOR (TTL-сумісна)



Логічний елемент XOR



Логічна функція **Виключне АБО (XOR)** — це дуже корисна схема, яку можна застосовувати в багатьох різних типах обчислювальних пристроїв.

"Виключне АБО" (Exclusive-OR, XOR) — це ще один тип цифрового логічного елемента, який широко застосовується в **арифметичних операціях**, оскільки може використовуватися для **отримання суми двох двійкових чисел**, а також у **схемах виявлення та виправлення помилок**.

У попередніх уроках ми бачили, що, використовуючи **три основні елементи** — **AND**, **OR** та **NOT**, — можна побудувати багато інших типів логічних функцій, наприклад, **NAND**, **NOR** або будь-яку іншу цифрову логічну схему.

Але існують ще два типи цифрових логічних елементів, які хоча й не є базовими (оскільки будуються шляхом комбінації інших елементів), проте їх **вихідна булева функція** настільки важлива, що їх розглядають як **повноцінні логічні елементи**. Ці два "гібридні" логічні елементи — це **Виключне АБО (Ex-OR)** та його доповнення **Виключне НЕ-АБО (Ex-NOR)**.

Раніше ми розглядали, що для **2-входового елемента OR**, якщо **A = "1"** або **B = "1"**, або **A і B = "1"**, тоді вихід також має бути **"1"**. Через це така логічна операція називається **інклюзивним АБО (Inclusive-OR)**, бо вона **включає випадок**, коли **Q = "1"** навіть тоді, коли **обидва входи A і B одночасно дорівнюють "1"**.

Якщо ж на виході з'являється **логічна "1"** лише тоді, коли **тільки A = "1"** або **тільки B = "1"**, але **не обидва одночасно**, тобто при двійкових вхідних комбінаціях **"01"** або **"10"**, то такий вихід також дорівнює **"1"**. Цей тип елемента називається **Виключне АБО** (*Exclusive-OR*, скорочено **Ex-OR**), оскільки його **булевий вираз виключає випадок**

Q = "1" коли **A і B одночасно дорівнюють "1"**.

Іншими словами, **вихід елемента XOR** переходить у **"HIGH"** тільки тоді, коли **два його входи мають різні логічні рівні**.

Властивість:
Непарна кількість логічних "1" на входах дає логічну "1" на виході.
Ці два входи можуть бути як на рівні **"1"**, так і на рівні **"0"**, що дає булевий вираз:

$Q = (\oplus B) = \overline{A}.B + A.\overline{B}$, $\oplus$ - читається як "виключне або" XOR

Де використовується XOR

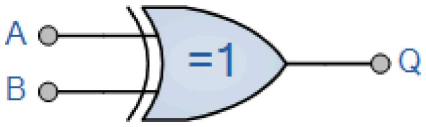
Функцію **Exclusive-OR** реалізують шляхом комбінації стандартних логічних елементів, утворюючи складніші функції, які **широко застосовуються** у:

- побудові **арифметико-логічних пристроїв (ALU)**
- **логічних компараторах**
- схемах **виявлення помилок**.

Двовходовий елемент **XOR** по суті є **суматором за модулем два**, оскільки він дає **суму двох двійкових чисел**. Через це він має складнішу реалізацію, ніж базові типи логічних елементів. Таблиця істинності, логічний символ та схема реалізації **2-входового XOR** зазвичай подаються у довідниках і виглядають відповідно.

Цифровий логічний елемент "Виключне АБО" (Ex-OR)

Позначення	Таблиця істинності

<table>
<tr><td rowspan="5"> 

2-входовий елемент Виключне АБО (Ex-OR)</td><td>B</td><td>A</td><td>Q</td></tr>
<tr><td>0</td><td>0</td><td>0</td></tr>
<tr><td>0</td><td>1</td><td>1</td></tr>
<tr><td>1</td><td>0</td><td>1</td></tr>
<tr><td>1</td><td>1</td><td>0</td></tr>
<tr><td>Булевий вираз $Q = A \oplus B$</td><td colspan="3">Читається як: A або B але НЕ ОБИДВА дає Q</td></tr>
</table>

Булевий вираз:

$$Q = \overline{A}B + A\overline{B}$$

Таблиця істинності показує, що **вихід елемента Exclusive-OR** стає **"1" (HIGH)** лише тоді, коли **два його входи знаходяться на різних логічних рівнях**. Якщо обидва входи **A** і **B** одночасно дорівнюють **"1"** або **"0"**, вихід дорівнює **"0"**. Через це його іноді називають *"елемент непарності"*, адже на виході **"1"** з'являється, коли **кількість одиниць на входах непарна**.

Ця здатність елемента **XOR** порівнювати два логічних рівні та формувати вихід, залежно від умови на вході, дуже корисна в **обчислювальних логічних схемах**.

Булевий вираз:

$$Q = (A \otimes B) = \overline{A}B + A\overline{B}$$

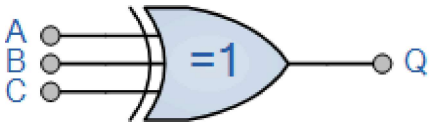
Логічна функція, яку реалізує **2-входовий XOR**, описується як:

"A або B, але не обидва" дає на виході Q.

У загальному випадку **XOR** дає **"1"**, якщо на входах **непарна кількість одиниць**. Якщо ж числа на входах рівні — вихід **"0"**.

⚠️ Якщо у функції **XOR** більше ніж два входи, то вона називається **"функція непарності"** (*odd function*) або **сума за модулем два (Mod-2-SUM)**, але вже не вважається "чистим" елементом XOR. Цей принцип можна поширити на будь-яку кількість входів, наприклад, для **3-входового XOR**.

3-входовий логічний елемент XOR

<table>
<tr><th>Позначення</th><th colspan="4">Таблиця істинності</th></tr>
<tr><td rowspan="9"> 

3-входовий логічний елемент виключне АБО (Ex-OR)</td><td>C</td><td>B</td><td>A</td><td>Q</td></tr>
<tr><td>0</td><td>0</td><td>0</td><td>0</td></tr>
<tr><td>0</td><td>0</td><td>1</td><td>1</td></tr>
<tr><td>0</td><td>1</td><td>0</td><td>1</td></tr>
<tr><td>0</td><td>1</td><td>1</td><td>0</td></tr>
<tr><td>1</td><td>0</td><td>0</td><td>1</td></tr>
<tr><td>1</td><td>0</td><td>1</td><td>0</td></tr>
<tr><td>1</td><td>1</td><td>0</td><td>0</td></tr>
<tr><td>1</td><td>1</td><td>1</td><td>1</td></tr>
<tr><td>Булевий вираз $Q = A \oplus B \otimes C$</td><td colspan="4">Читається як: "Будь-яка непарна кількість входів" дає Q</td></tr>
</table>

Булевий вираз:

$$Q = \overline{A}BC + A\overline{B}C + \overline{ABC}$$

Символ, що використовується для позначення **Exclusive-OR (функції непарності)**, трохи відрізняється від символу звичайного елемента **Inclusive-OR**. Для логічного елемента OR булевий вираз — це **логічне додавання**, яке позначається стандартним **знаком “+”**.

Для **Exclusive-OR** використовують знак **плюс у колі (⊕)**. Цей символ також застосовується у математиці для позначення **прямої суми підоб’єктів**. Отже, запис для функції Exclusive-OR:

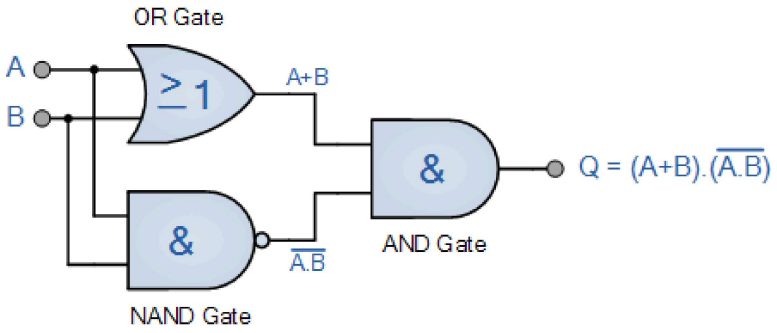
A ⊕ B

Раніше ми вже говорили, що **XOR** не є базовим логічним елементом, а утворюється комбінацією кількох стандартних елементів. Використовуючи таблицю істинності для 2-входового варіанта, можна переписати функцію XOR так:

$$(A + B) \cdot (\overline{A} + \overline{B})$$

Це означає, що її можна реалізувати, комбінуючи окремі **елементи OR, AND та NOT**.

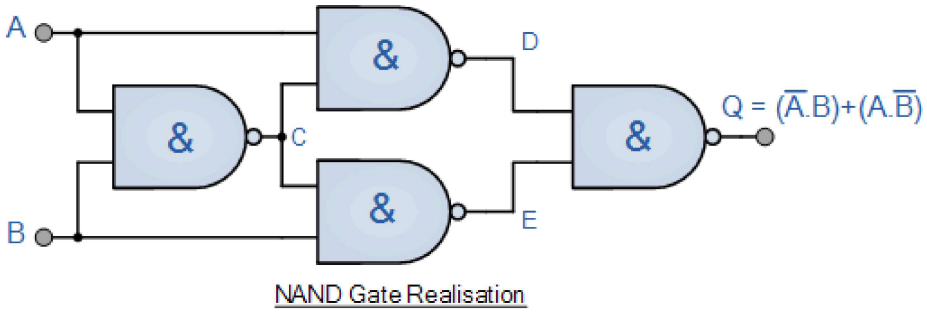
Еквівалентна схема елемента Ex-OR — це з'єднання кількох базових логічних елементів, яке дає ту саму логічну поведінку, що і XOR.



Одним із головних недоліків реалізації функції **Ex-OR** у наведеному вище варіанті є те, що в ній використовуються **три різні типи логічних елементів** — **OR**, **NAND** та **AND**.

Більш простий спосіб побудувати функцію **Ex-OR** за допомогою одного типу елемента — це використати наш старий "улюблений" **елемент NAND**, як показано нижче.

Реалізація функції Ex-OR за допомогою NAND-елементів



Елементи **Exclusive-OR (XOR)** переважно використовуються для побудови схем, які виконують **арифметичні операції та обчислення**, особливо в **суматорах** і **напівсуматорах**, оскільки вони можуть забезпечувати функцію **біта переносу** або працювати як **керований інвертор**, де один вхід передає двійкові дані, а на інший подається керуючий сигнал.

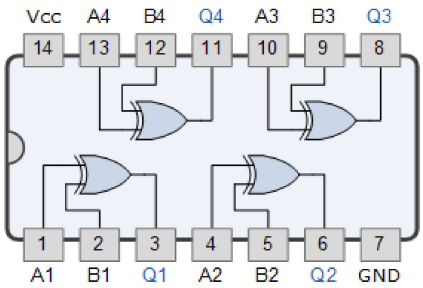
Поширені мікросхеми логічних елементів Exclusive-OR

TTL-логіка:

- **74LS86** — чотири 2-входові XOR

CMOS-логіка:

- **CD4030** — чотири 2-входові XOR
- **7486** — чотири 2-входові XOR (TTL-сумісна)

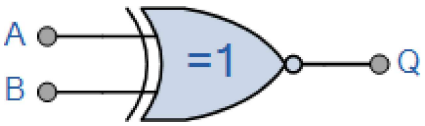


Логічна функція **Exclusive-OR (XOR)** — це дуже корисна схема, яку можна застосовувати в багатьох типах обчислювальних пристроїв. Хоча вона й не є базовим логічним елементом, її **універсальність** і **практична цінність** зробили її **стандартною логічною функцією** зі своїм власним **булевим виразом, оператором та символом**. Елемент XOR широко доступний у вигляді стандартної мікросхеми **чотирьох 2-входових елементів 74LS86 (TTL)** або **4030B (CMOS)**.

Одне з найпоширеніших застосувань — це **базовий логічний компаратор**, який видає **логічну “1”**, коли два вхідних біти **не рівні**. Через це елемент XOR має статус **функції непарності** (*odd function*).

Для порівняння чисел, що містять два або більше бітів, потрібні додаткові елементи XOR. Наприклад, **логічний компаратор 74LS85** працює з **4-бітними числами**.

Логічний елемент XNOR



Функція **Exclusive-NOR (XNOR)** — це цифровий логічний елемент, який є **зворотною (доповнювальною) формою** функції **Exclusive-OR (XOR)**.

По суті, **XNOR** — це комбінація елемента **XOR** та інвертора **NOT**, але його таблиця істинності схожа на таблицю елемента **NOR** тим, що вихід зазвичай перебуває на **логічному рівні "1"** і переходить у **логічний "0"**, коли будь-який із входів має логічний рівень "1".

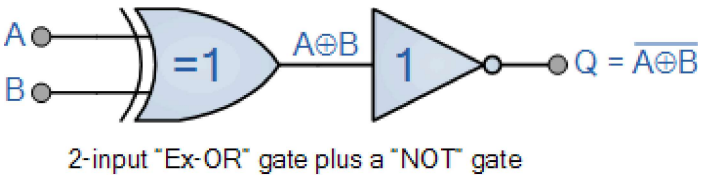
Однак вихід **"1"** з'являється тільки тоді, коли **обидва входи** перебувають **на одному логічному рівні** — або **"00"**, або **"11"**. У такому випадку булевий вираз буде:

$$Q = \overline{(A \oplus B)} = \overline{A}.\overline{B} + A.B$$

Отже, вихід **XNOR** стає **HIGH**, коли входи **A** і **B однакові** — обидва "1" або обидва "0". Іншими словами: **парна кількість "1" на входах → вихід "1"**, в усіх інших випадках — вихід "0". Через цю властивість XNOR іноді називають **елементом еквівалентності (Equivalence Gate)**, оскільки він видає "1", коли входи **логічно рівні**.

Логічний символ елемента **Exclusive-NOR (XNOR)** — це символ **Exclusive-OR (XOR)** з **колом ("бульбашкою інверсії")** на виході, яке позначає функцію **NOT**. Отже, логічний елемент **XNOR** є **зворотною (доповнювальною) формою** елемента **Exclusive-OR (A ⊕ B)**, який ми розглядали раніше.

Еквівалент елемента Ex-NOR

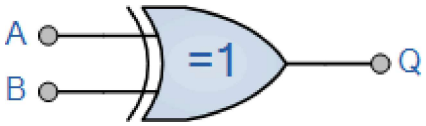


2-input "Ex-OR" gate plus a "NOT" gate

Функція **Exclusive-NOR**, яку також записують як **“Ex-NOR”** або **“XNOR”**, реалізується шляхом комбінації стандартних логічних елементів для утворення складніших логічних функцій. Нижче наведено приклад **2-входового елемента Exclusive-NOR**.

Цифровий логічний елемент “Exclusive NOR” (XNOR)

2-входовий елемент Exclusive-NOR (XNOR)

<table>
<tr><th>Позначення</th><th colspan="3">Таблиця істинності</th></tr>
<tr><td rowspan="6"> 

2-входовий елемент Exclusive-NOR (XNOR)</td><td>B</td><td>A</td><td>Q</td></tr>
<tr><td>0</td><td>0</td><td>1</td></tr>
<tr><td>0</td><td>1</td><td>0</td></tr>
<tr><td>1</td><td>0</td><td>0</td></tr>
<tr><td>1</td><td>1</td><td>1</td></tr>
<tr><td>Булевий вираз $Q = \overline{A \oplus B}$</td><td colspan="3">Читається як: "Якщо A і B однакові, тоді Q"</td></tr>
</table>

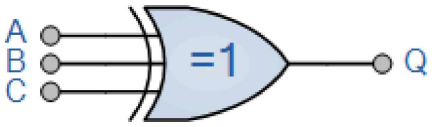
Подано булевий вираз: **Q = A B + Ā B̄**

Логічна функція, реалізована 2-входовим елементом Ex-NOR, формулюється так: **«Коли A і B однакові, на виході Q буде логічна 1»**. Загалом, елемент **Exclusive-NOR** дає вихідне значення логічної **1 тільки тоді**, коли на його входах парна кількість логічних 1 (це інверсія роботи елемента Ex-OR), **за винятком** випадку, коли всі його входи перебувають у стані LOW.

Функція Ex-NOR із більш ніж двома входами називається **парною функцією** або **modulo-2-sum (Mod-2-SUM)**, а не Ex-NOR. Це визначення можна поширити на будь-яку кількість окремих входів, як показано нижче для 3-входового елемента Exclusive-NOR.

3-входовий елемент Exclusive-NOR (XNOR)

Позначення	Таблиця істинності



3-входовий логічний елемент Exclusive-NOR (XNOR)

C	B	A	Q
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Булевий вираз $Q = \overline{A \oplus B \otimes C}$

Читається як: «Будь-яка ПАРНА кількість входів дає Q»

Подання у вигляді булевого виразу:

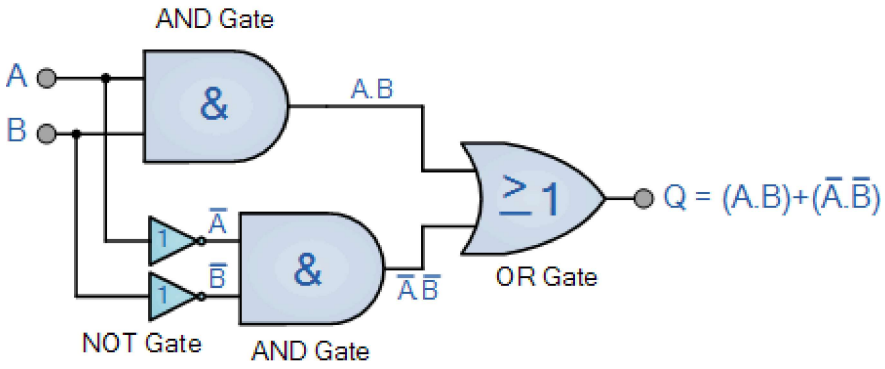
$$Q = \overline{ABC} + AB\overline{C} + A\overline{B}C + \overline{A}BC$$

Раніше ми зазначали, що функція **Ex-NOR** є комбінацією двох базових логічних елементів — **Ex-OR** та **NOT**. Використовуючи таблицю істинності для 2-входового елемента, ми можемо розкласти функцію **Ex-NOR** так:

$$Q = \overline{A \oplus B} = (A.B) + (\overline{A}.\overline{B})$$

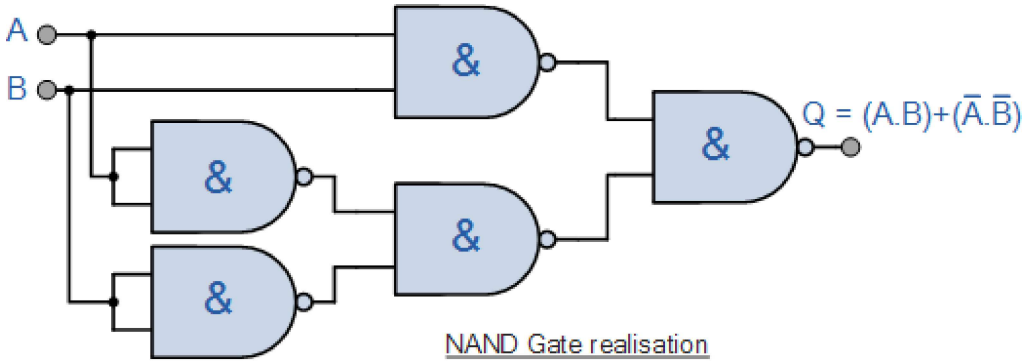
Це означає, що реалізувати цю функцію можна, використовуючи окремі логічні елементи, як на відповідній схемі.

Еквівалентна схема логічного елемента XNOR (Ex-NOR)



Одним із головних недоліків реалізації наведеної вище функції Ex-NOR є те, що в її базовій схемі використовується три різних типи логічних елементів: AND, NOT і, зрештою, OR. Простіший спосіб побудувати функцію Ex-NOR з використанням лише одного типу логічних елементів — це застосувати елементи NAND, як показано нижче.

Реалізація функції Ex-NOR за допомогою елементів NAND



Елементи **Ex-NOR** здебільшого застосовуються в електронних схемах, що виконують арифметичні операції та перевірку даних, таких як **суматори, віднімачі або перевірювачі парності**. Оскільки елемент Ex-NOR видає логічний рівень **“1”**, коли обидва його входи рівні, його можна використовувати для **порівняння величини двох бінарних цифр або чисел**, тому елементи Ex-NOR застосовуються в схемах **цифрових компараторів**.

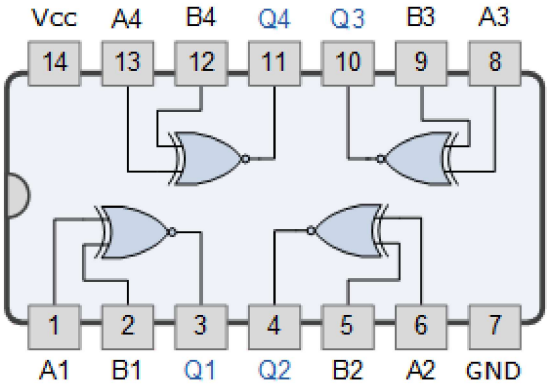
Приклади доступних ІМС з елементами Ex-NOR:

TTL Logic Ex-NOR:

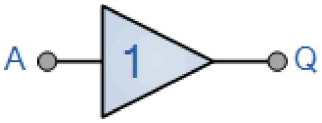
74LS266 – чотири 2-входові елементи в одному корпусі

CMOS Logic Ex-NOR:

- **CD4077** – чотири 2-входові елементи в одному корпусі
- **74266** – чотири 2-входові Ex-NOR елементи



Цифровий буфер



Цифрові буфери та тристанові буфери можуть забезпечувати підсилення струму в цифровій схемі для керування вихідними навантаженнями.

Цифровий буфер — **логічний елемент, протилежний інвертору (NOT)**, про який ми говорили в попередньому уроці, де було показано, що вихідний стан інвертора є **доповненням, протилежністю або інверсією** його вхідного сигналу.

Наприклад, коли єдиний вхід інвертора має стан **“HIGH”**, його вихідний стан **не буде “HIGH”**. Коли вхідний сигнал **“LOW”**, його вихідний стан **не буде “LOW”**, іншими словами, він **інвертує** свій вхідний сигнал, звідси і назва **«інвертор»**.

Але іноді в цифрових електронних схемах нам потрібно **ізолювати логічні елементи один від одного** або **забезпечити керування навантаженнями більшої потужності**, ніж звичайні, такими як реле, соленоїди та лампи, **без необхідності інверсії сигналу**. Один із типів логічних елементів з одним входом, який дозволяє нам це робити, називається **цифровий буфер**.

На відміну від інвертора або NOT з одним входом і одним виходом, наприклад **TTL 7404**, який **інвертує або доповнює** свій вхідний сигнал на виході, **буфер не виконує інверсії** або **прийняття рішень** (як логічні елементи з двома або більше входами), а замість цього **відтворює на виході точно такий же стан, як і на вході**. Іншими словами, **цифровий буфер не змінює сигнал** — його вихідний стан **збігається зі станом входу**.

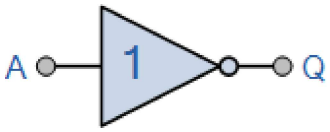
Отже, **цифрові буфери можна розглядати як ідемпотентні елементи**, що застосовують **ідемпотентний закон Буля**, оскільки при проходженні сигналу через цей пристрій його значення **не змінюється**. Тобто цифровий буфер є **неінверсним пристроєм** і має такий булів вираз: **Q = A**.

Тоді логічну операцію цифрового буфера з одним входом можна визначити так:

«Q істинне тільки тоді, коли A істинне».

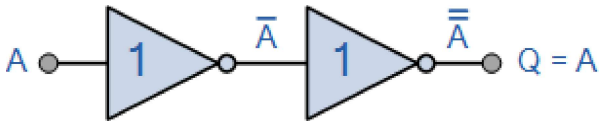
Іншими словами, **вихідний стан Q** буфера є істинним (**лог. “1”**) тільки тоді, коли його вхід **A** є істинним, інакше вихід є хибним (**лог. “0”**).

Одновхідний цифровий буфер

<table>
<tr><th>Позначення</th><th colspan="2">Таблиця істинності</th></tr>
<tr><td rowspan="3"> 

Цифровий буфер</td><td>A</td><td>Q</td></tr>
<tr><td>0</td><td>0</td></tr>
<tr><td>1</td><td>1</td></tr>
<tr><td>Булевий вираз Q = A</td><td colspan="2">Читається як: A дає Q</td></tr>
</table>

Подвійне інвертування за допомогою логічних елементів NOT



Ви можете подумати: *“Який сенс у цифровому буфері, якщо він не інвертує і не змінює вхідний сигнал жодним чином, та не виконує жодних логічних операцій, як елементи AND або OR? Чому б просто не використати шматок дроту?”* — і це справедливе зауваження.

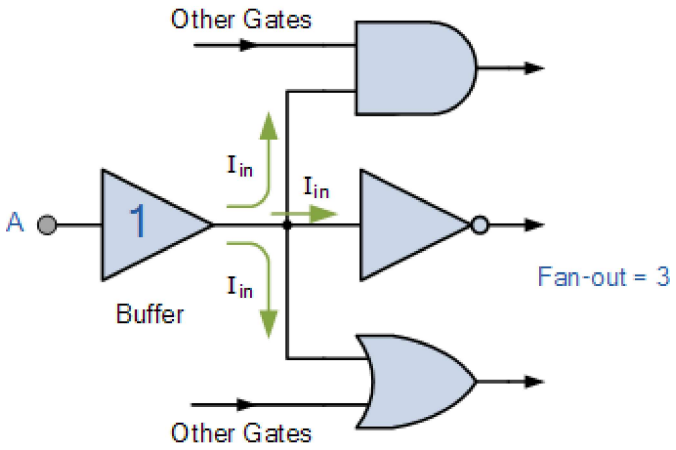
Але **неінвертуючий цифровий буфер** має багато застосувань в цифровій електроніці, і одна з його головних переваг — це **цифрове підсилення**.

Буфери можна використовувати для **ізоляції** інших логічних елементів або каскадів схеми, запобігаючи впливу імпедансу одного кола на імпеданс іншого. Також цифровий буфер можна застосовувати для **керування навантаженнями з великим струмом**, наприклад, транзисторними ключами, оскільки їх здатність віддавати струм на виході зазвичай набагато вища, ніж потреби вхідного сигналу.

Іншими словами, **буфери можуть виконувати роль підсилювачів потужності цифрового сигналу**, адже вони мають так звану високу **fan-out** здатність (можуть керувати великою кількістю входів інших логічних елементів).

Приклад Fan-out для цифрового буфера

(fan-out означає "розгалуження" або "віялоподібний вихід")



Fan-out параметр буфера (або будь-якої цифрової IC) — це **здатність виходу керувати певну кількість навантажень** або **вихідна струмова здатність** логічного елемента, що дає більшу потужність сигналу на виході порівняно з входом.

Іноді необхідно підключити більше ніж один логічний елемент до виходу іншого або комутувати навантаження з великим струмом, наприклад **світлодіод**. У такому випадку **буфер дозволяє це зробити**.

Зазвичай вихід логічного елемента підключають до входів інших елементів. **Кожен вхід споживає певний струм** від виходу для зміни свого стану, і кожне додаткове підключення збільшує навантаження.

Отже, **fan-out** — це **кількість паралельних навантажень, які можна одночасно підключити до виходу одного цифрового буфера або логічного елемента**. Як джерело струму, буфер може мати fan-out до **20 входів** тієї ж логічної сім'ї.

Якщо цифровий буфер має **високий fan-out** (джерело струму), він також повинен мати і **високий fan-in** (приймач струму). Однак **час затримки поширення сигналу** помітно погіршується зі збільшенням fan-in, тому **слід уникати елементів із fan-in більше ніж 4**.

Існує обмеження на кількість входів і виходів, які можна з'єднувати між собою. У випадках, коли потрібно **розв'язати (ізолювати) логічні елементи один від одного**, використовують **тристабільний буфер** (*Tri-state Buffer*) або **драйвер з трьома станами**.